CONDICIONAIS TEST, [[ ]] E IF

*O script de rotação tinha uma guarda impecável: `if [ -f $ARQUIVO ]`. Na noite em que `ARQUIVO` ficou vazio, o teste **passou** — e a condição que existia para impedir o estrago disse "sim, o arquivo existe" sobre um nome vazio.*

055
GUIAS LINUX

`[` NÃO É SINTAXE: É UM COMANDO — E ISSO EXPLICA TUDO

`[ -f $ARQUIVO ]` → `[` `-f` `]` — **1 argumento útil.** A regra do POSIX para um argumento é "verdadeiro se a cadeia não for vazia" — e `-f` não é vazia. **Passa.**

`[ -f "$ARQUIVO" ]` → `[` `-f` `` ` ` `` `]` — **2 argumentos.** Agora é mesmo um teste de arquivo, sobre um nome vazio. **Falha**, como deveria.

`[[ -f $ARQUIVO ]]` → PALAVRA-CHAVE DO SHELL — **Não é um comando.** O shell analisa o conteúdo *sem* dividir em palavras — as aspas ficam dispensáveis. **Falha**, como deveria.

Quando o `[` é finalmente executado, **não existe mais vestígio de que houve ali uma variável**: a linha já passou pelas seis etapas de expansão e a variável vazia sem aspas simplesmente sumiu antes da chamada. Daí as duas regras: **aspas em toda expansão dentro de `[ ]`**, e **`[[ ]]` sempre que o script for Bash**. Pelo mesmo motivo, os espaços em volta dos colchetes não são opcionais — `[-f arq]` é o shell procurando um comando chamado `[-f`, e o `]` é só o último argumento obrigatório. E o `if` não avalia expressão booleana: ele **executa um comando e olha o código de saída** (zero = verdadeiro), então `if grep -q '^root:' /etc/passwd`, `if systemctl is-active --quiet ssh` e `if command -v nmap >/dev/null` dispensam colchete nenhum — e são melhores que capturar a saída e testá-la.

O MAPA DOS OPERADORES — ARQUIVO · CADEIA · NÚMERO

ARQUIVO

`-e` existe (qualquer tipo)
`-f` arquivo comum · **`-d`** diretório
`-L` link simbólico · **`-s`** não vazio
`-r -w -x` legível, gravável, executável
`-u -g -k` SUID, SGID, *sticky*
`-b -c -p -S` bloco, char, fifo, socket
`a -nt b` a é mais novo · **`a -ef b`** mesmo inode

`-f` segue o link e responde sobre o **destino**; `-L` responde sobre o próprio link.

CADEIA

`-z s` vazia · **`-n s`** não vazia
`a == b` igual — **padrão** no lado direito
`a != b` diferente
`a =~ re` casa com a expressão regular
`a < b` ordem alfabética

Em `[[ ]]`, **cite o lado direito** do == para comparar literalmente. Com =~ é o inverso: a regex **não** vai entre aspas — guarde-a numa variável e leia o resultado em `${BASH_REMATCH[1]}`.

NÚMERO

`-eq` igual · **`-ne`** diferente
`-lt -le` menor, menor ou igual
`-gt -ge` maior, maior ou igual
Dentro de **`(( ))`**, os símbolos:
`(( a > b ))` · **`(( a ≥ 1 && a ≤ 10 ))`**
`(( contador++ ))` · **`(( total += v ))`**

`[[ 10 > 9 ]]` é falso: compara texto, e "1" vem antes de "9".

A regra que não falha: número dentro de `(( ))`, o resto dentro de `[[ ]]`. Os operadores com hífen funcionam nos dois colchetes; os símbolos só têm sentido numérico em `(( ))` — e trocar os contextos é o erro **silencioso** mais caro do capítulo: `[[ $versao > 9 ]]` aprova a versão 10 como menor que a 9 e nunca reclama. Guarde `[ ]` só para script que precise rodar sob `/bin/sh`; ali, use `=` (não `==`) e troque o obsoleto `-a` / `-o` por `[ … ] && [ … ]`.

kali@kali: ~/lab-if

```
$ re='^([0-9]{1,3}\.){3}[0-9]{1,3}(/[0-9]{1,2})?$'   # regex SEM aspas no teste
$ [[ $alvo =~ $re ]] || { erro "alvo invalido: $alvo"; exit 1; }
$ [[ $arq == *.log ]] && echo casou     # padrao glob, nao regex
$ case "$1" in
      -h|--help)      ajuda; exit 0 ;;
      *.tar.gz|*.tgz) tar xzf "$1" ;;
      [0-9]*)         echo "comeca com digito" ;;
      "")             erro "argumento vazio"; exit 1 ;;
      *)              erro "opcao desconhecida: $1"; exit 1 ;;   # o *) e obrigatorio
    esac
$ ( set -e; n=0; (( n++ )); echo "nao chega aqui" )  # ABORTA: devolveu o 0 antigo
$ ( set -e; n=0; (( ++n )); echo "n=$n" )            n=1  # ou: (( n++ )) || true
```

Testar um arquivo não é mecanismo de segurança. Entre `if [[ -w "$ALVO" ]]` e o `rm -rf -- "$ALVO"` da linha seguinte existe um intervalo em que outro processo troca o arquivo por um link apontando para outro lugar — é o **TOCTOU** (*time of check to time of use*), explorável quando o script roda com privilégio sobre caminho gravável por terceiros, com o `/tmp` como caso clássico. Defesas: **aja e trate o erro** em vez de testar antes (`rm -rf -- "$ALVO" 2>/dev/null || true` não tem intervalo) · `tmp="$(mktemp -d)"` para temporário, que já nasce `700` e com nome imprevisível, nunca um nome previsível · `trap` para limpar na saída · e **nunca** opere como root sobre caminho de terceiro sem `--no-dereference` ou `-not -type l` . O teste de arquivo continua útil — para dar mensagem de erro decente antes de tentar.

RESUMO RÁPIDO

- ✓ **`[` é um comando**: espaços obrigatórios, `]` é só o último argumento, e o conteúdo passa pelas seis etapas de expansão **antes** do teste.
- ✓ **`if` executa um comando e olha o código de saída.** Zero é verdadeiro — qualquer programa serve de condição.
- ✓ **Em Bash, `[[ ]]` é o padrão:** não divide em palavras, aceita `&&` / `||` , casa padrão com `==` e regex com `=~` (preenchendo `BASH_REMATCH`).
- ✓ **Número em `(( ))` , o resto em `[[ ]]` .** `[[ 10 > 9 ]]` é falso.
- ✓ **`case` para várias alternativas**, com `*)` sempre presente — sem ele, entrada inesperada passa em silêncio.
- ✓ **Sob `set -e` , `(( n++ ))` com `n=0` derruba o script.** Use `(( ++n ))` .

NO KALI

Validar o alvo com `=~` antes de disparar a ferramenta é a diferença entre laboratório e incidente. Um teste de formato — `[[ $alvo =~ ^[0-9]{1,3}(\.[0-9]{1,3}){3}(/[0-9]{1,2})?$ ]]` — pega o alvo digitado errado *antes* de a varredura sair; melhor ainda é conferir contra a lista do escopo autorizado e recusar o que não estiver nela. Varrer fora do escopo contratado é crime pela Lei 12.737/2012, e um `if` de três linhas é a barreira mais barata que existe contra um erro de digitação. **Desde 2020 o usuário padrão é o `kali` , não o root**: um `(( EUID ≠ 0 )) && { erro "execute com sudo"; exit 1; }` na primeira linha economiza o trabalho perdido no meio do caminho. **Num *rolling*, confira se a ferramenta existe:** `command -v nmap >/dev/null || { erro "sudo apt install nmap"; exit 1; }` — o `which` não é embutido, varia entre distribuições e devolve códigos inconsistentes. E cuidado com ponto de montagem que some: em mídia montada pelo `udisks` ou em live USB, um `[[ -d /media/kali/EVIDENCIA ]]` pode passar e o script gravar no diretório vazio do ponto de montagem em vez de na mídia — `mountpoint -q` responde à pergunta certa. Por fim, `-r` e `-w` respondem sobre a permissão **efetiva**, ACLs e *capabilities* incluídas: um `[[ -r /etc/shadow ]]` verdadeiro para usuário comum é achado de auditoria.